


# Road-Constrained Vehicle Trajectory Recovery from Traffic Video Using Spatio-Temporal Voxel Representation

Taihang Dong
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, Zhejiang, China
taihangd@zju.edu.cn

Jun Zhang
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, Zhejiang, China
zj.cs@zju.edu.cn

Ping Chen*
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, Zhejiang, China
zjuchenping@zju.edu.cn

Rongkai Wang
School of Electrical Engineering and Automation, Fuzhou University
Fuzhou, Fujian, China
rk_wang@fzu.edu.cn

Yu Fu†
School of Information Science and Engineering, Lanzhou University
Lanzhou, Gansu, China
fuyu@lzu.edu.cn

Dingyu Yang
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, Zhejiang, China
yangdingyu@zju.edu.cn

## Abstract

Road-constrained trajectory recovery from traffic surveillance videos has become a critical component in applications ranging from intelligent transportation to urban planning. Existing approaches typically perform explicit camera-road matching, which either relies on labor-intensive camera calibration or suffers from matching errors. Moreover, prior methods often represent trajectories as sequences of tuples (e.g., node or edge with timestamps), which limits effective spatio-temporal mining, leading to suboptimal recovery accuracy. To address these problems, we propose a novel trajectory recovery framework based on spatio-temporal voxel representation. The real-world coordinates of trajectory nodes are encoded as voxel indices, which naturally capture spatio-temporal relationships. Our framework leverages this voxel representation to enable direct road-constrained trajectory recovery from camera observations, eliminating the need for explicit heuristic camera-road matching. Building upon this voxel formulation, we further present a shape-aware evaluation scheme that assesses trajectories within a spatio-temporal voxel grid, capturing edge and structural information to provide a complementary perspective to existing evaluation metrics. Finally, extensive experiments across multiple datasets demonstrate that our method achieves state-of-the-art trajectory recovery accuracy, with an average F1-score improvement of 14.7%.

## CCS Concepts

• **Information systems** → **Spatial-temporal systems**; **Data mining**.

*Ping Chen is also affiliated with the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, Zhejiang, China.
†Yu Fu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
*KDD'26, Jeju, Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817633

## Keywords

Vehicle Trajectory Recovery, Spatio-Temporal Modeling, Traffic Video Analysis

**ACM Reference Format:**
Taihang Dong, Jun Zhang, Ping Chen, Rongkai Wang, Yu Fu, and Dingyu Yang. 2026. Road-Constrained Vehicle Trajectory Recovery from Traffic Video Using Spatio-Temporal Voxel Representation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817633

**Resource Availability:**
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.20372835 and https://github.com/taihangd/RETRIEVE.

## 1 Introduction

With the proliferation of large-scale video surveillance systems, traffic cameras have become a pervasive data source for intelligent transportation and urban governance, continuously capturing vehicle activities across urban road networks [9]. The widespread deployment of traffic cameras generates massive volumes of video data, enabling the recovery of complete vehicle trajectories at the city scale. Unlike other sensing modalities such as GPS, which may be unavailable for many vehicles, video surveillance offers broad and passive coverage of all targets within a camera's field of view [5]. In this context, trajectory recovery from traffic cameras supports a wide range of downstream applications, including traffic flow analysis [14, 16, 26] and co-movement pattern mining [2, 34], and public security tasks like suspect vehicle tracing [15, 30].

Specifically, the trajectory recovery task aims to reconstruct complete vehicle trajectories on a road network from city-scale multi-camera videos along with camera and road network information, as illustrated in Figure 1(a). To this end, existing trajectory recovery methods [18, 22, 29, 32, 33] typically follow a similar process. First, they cluster snapshots based on visual cues, license plate information, or other distinguishing features. Then, for each cluster, they map snapshots to corresponding road nodes or edges.

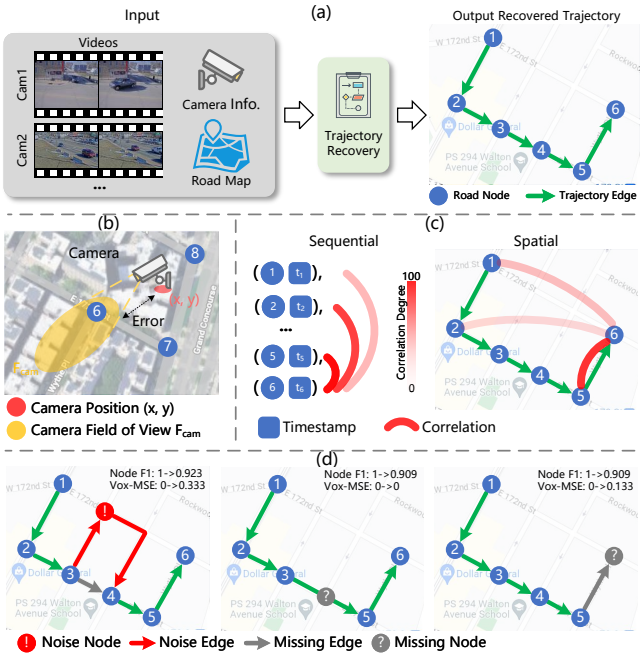



**Figure 1: The trajectory recovery task and problems. (a) the overview of the trajectory recovery task, (b) camera-road matching error, (c) sequence representation characteristics, (d) trajectory evaluation metric characteristics.**

Subsequently, they model the transition probabilities between different road segments, or rely on prior assumptions to reconstruct complete road network trajectories between road nodes or edges.

These methods generally do not account for camera-road matching errors. Prior methods [22, 29, 32, 33] attempt to resolve this by directly matching camera nodes to road network nodes or edges, either through labor-intensive manual annotation or error-prone rule-based heuristics. Specifically, a camera's physical position does not necessarily coincide with the road regions covered by its field of view, and assigning a camera to a road node solely based on its physical position inevitably introduces errors. A fundamental ambiguity arises because a camera's field of view typically spans a region of the road network, making its assignment to a single node or edge an inherently coarse approximation. As illustrated by the camera in Figure 1(b), the camera's physical location $(x, y)$ often does not correspond to the actual road region $F_{cam}$ captured in the video, and representing the field of view using a single road node (e.g., node 6) remains a simplifying approximation. Even with techniques that enable more precise vehicle localization, modeling a camera by its field-of-view region is still an intuitive approximation. For example, to simulate the impact of such errors on trajectory recovery, we inject uniform random noise of up to 20 meters into the camera locations in Section 5.5. Under this perturbation, the F1-score of MMVC+ [32] drops by 7.1%. This naturally leads us to the question: **How can we mitigate the impact of camera-road matching errors on recovery performance?**

In addition, these methods operate directly on sequential trajectory representations and rely heavily on discrete numerical formulations, making them highly sensitive to the indexing order of nodes in the sequence. For example, classical methods [7, 20, 23] model trajectories as spatio-temporal node sequences and characterize movement behavior via transition probabilities between nodes. As illustrated in Figure 1(c), correlations between tuples in a sequential representation are influenced by their order indices and thus tend to assign stronger relationships to tuples that are closer in order. For example, the correlation between $(6, t_6)$ and $(2, t_2)$ is regarded as stronger than that between $(6, t_6)$ and $(1, t_1)$. However, in physical space, this assumption may not hold, as tuples that are adjacent in the sequence can be far apart spatially. This mismatch reveals the limitation of sequence-based representations in reflecting true geometric relationships. This leads us to the question: **How can we design more effective trajectory representations that better mine spatio-temporal information?**

To address these problems, we propose the solution RETRIEVE (Road-constrained vEhicle Trajectory Recovery using spatIo-tEmporal Voxel rEpresentation), which avoids explicit camera-road matching and instead models spatio-temporal relationships using the voxel representation. Motivated by the First Law of Geography [21], we encode raw spatio-temporal camera positions with timestamps into the three-dimensional (3D) spatio-temporal voxel representation. In this representation, the original continuous spatial coordinates and timestamps are encoded as voxel indices. This representation more naturally captures spatio-temporal relationships, facilitating effective spatio-temporal mining. This voxel representation is processed by a deep neural network that learns the trajectory recovery process from raw camera observations to road-constrained trajectories. To handle the growing volume of spatio-temporal data, we adopt a segment-fusion strategy that partitions camera observations into manageable segments, processes them independently with deep learning models, and fuses the results into complete trajectories via integer linear programming. This avoids processing empty voxels and thereby reduces resource overhead.

Building on the spatio-temporal voxel representation, we further investigate its application to trajectory evaluation. Common trajectory recovery metrics, such as Longest Common Subsequence (LCSS) [23] and Spatio-Temporal Longest Common Subsequence (STLC) [20], are **node-centric**, as they operate on sequences of discrete trajectory tuples. These metrics fail to capture variations in travel distance and transition characteristics between nodes, as they ignore trajectory edges and overall structure, leading to ambiguous assessments when trajectories contain noisy or missing nodes. For example, as illustrated by the cases in Figure 1(d), noisy nodes may introduce edge connection errors that are far more severe than the node errors themselves. Moreover, missing node 6 is more detrimental than missing node 4, since the entire connection between nodes 5 and 6 is lost, yet existing metrics are unable to capture this difference. To address these limitations, we introduce a spatio-temporal evaluation metric built upon the voxel representation, which represents trajectories in voxel form and measures their discrepancy. Unlike traditional metrics, our metric considers trajectory edges and structural information, providing a shape-aware assessment of trajectory recovery quality.

In summary, the key contributions of this paper are:

- We propose **RETRIEVE**, a novel trajectory recovery framework that integrates voxel representations with a segment-fusion strategy, enabling more accurate recovery of road-constrained trajectories from surveillance videos.
- We introduce the **spatio-temporal voxel** as a new data representation for trajectory recovery, which avoids explicit camera-road matching and allows more effective exploitation of spatio-temporal information.
- We design a **voxel-based evaluation metric** that inherently captures spatio-temporal trajectory characteristics, providing a shape-aware assessment.
- We implement RETRIEVE and conduct extensive experiments, demonstrating that it achieves superior trajectory recovery accuracy overall. The code is publicly available.

## 2 Problem Definition

**Definition 1 (Traffic Camera Input).** Let $C = \{c_1, c_2, \ldots\}$ be the set of surveillance cameras deployed in a city. Each camera $c \in C$ is associated with a fixed position $(x_c, y_c)$ in the real-world coordinate system and optional metadata (e.g., viewing direction). The input consists of the city-scale multi-camera video streams captured by $C$ together with the camera metadata.

**Definition 2 (Road Network).** The road network is modeled as a directed graph $G = (V, E)$, where $V$ denotes intersections and $E$ denotes directed road segments. Each node $rid \in V$ has a geographic coordinate $(x_{rid}, y_{rid})$, and each edge $e \in E$ may carry attributes such as length and road category.

**Definition 3 (Road-Constrained Trajectory).** A road-constrained vehicle trajectory is a time-ordered sequence $\mathcal{T} = \{(rid_i, t_i)\}_{i=1}^{K}$, where $rid_i \in V$ denotes a road node and $t_i$ is the corresponding time. $\mathcal{T}$ must follow the directed connectivity of $G$. Some existing methods represent trajectories as sequences of edge IDs, which are semantically equivalent and can be deterministically converted into this node-based formulation.

**Problem Statement.** Given the road network $G = (V, E)$, the camera set $C$, and the corresponding vehicle snapshot data $\mathcal{I}$ extracted from the videos, the goal is to recover the continuous and complete road-constrained trajectories $\mathcal{T}$ on $G$ from sparse and discrete observations in $\mathcal{I}$.

## 3 RETRIEVE

To mitigate matching errors and explore a more effective representation strategy, we propose RETRIEVE. As illustrated in Figure 2, the framework consists of the stages: 1) preprocessing and clustering, 2) candidate segmentation, 3) spatio-temporal voxelization, 4) model training/inference, 5) trajectory node extraction, and 6) trajectory segment fusion. Following prior work [32, 33], we extract visual features and license plate information from vehicle snapshot records and cluster snapshots based on visual similarity to associate observations of the same vehicle. Snapshot representatives are obtained using the method proposed in [8] and treated as candidate observations. (Details in Appendix A.1). We then construct the spatio-temporal voxel representation to encode camera observations and employ deep neural networks to infer trajectory voxel representations, from which sequential trajectory nodes are extracted. To handle large-scale data and fragmented observations, we design a dynamic programming-based candidate segmentation strategy, followed by a trajectory segment fusion module that assembles recovered segments into complete trajectories.

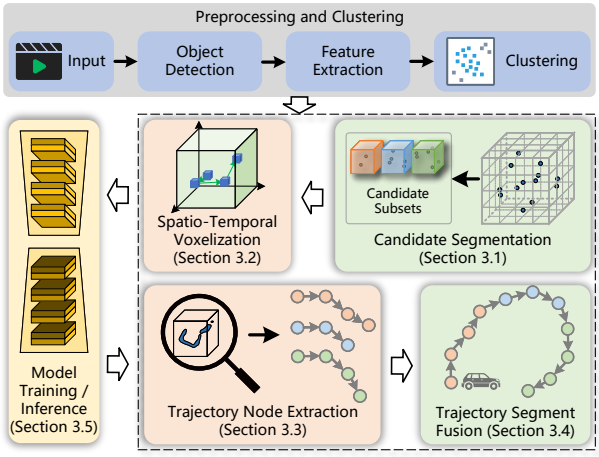



**Figure 2: The overview of RETRIEVE.**

### 3.1 Candidate Segmentation

After obtaining the clustering results, we adopt a segment-fusion strategy to cope with the continuously growing data volume and spatio-temporal coverage. Specifically, we first partition these candidates based on local spatio-temporal regions as shown in Figure 3, perform trajectory recovery independently within each region, and finally merge the recovered trajectories across regions in Section 3.4.

Since the optimal covering problem in three dimensions is known to be NP-hard [12], we propose an efficient heuristic-based segmentation method to achieve effective partitioning. Candidates are segmented according to spatio-temporal proximity. Candidates grouped into the same segment are required to fall within a spatio-temporal region, and the segmentation is further encouraged to produce spatio-temporal spans that are as close as possible to the target range. To efficiently implement this strategy, we formulate the segmentation process using dynamic programming. The state transition is defined as:

$$dp\,[i] = \min_{j \in [i_{\text{start}},\, i-1]} dp\,[j] + \text{Cost}_{\text{seg}}(j, i), \tag{1}$$

where $dp[i]$ denotes the minimum cumulative cost up to the $i$-th trajectory node, and $\text{Cost}_{\text{seg}}(j, i)$ represents the segmentation cost of forming a segment spanning from node $j$ to node $i$. The lower bound $i_{\text{start}}$ defines the earliest node considered in the look-back window during each update. To limit computation and maintain locality, we restrict the look-back window to 50 nodes, i.e., $i_{\text{start}} = \max(0,\ i - 50)$.

The segmentation cost between indices $(j, i)$ is defined as

$$\text{Cost}_{\text{seg}}(j, i) = \sum_{d \in \{x, y\}} g(r_d - \tau_d) + \lambda_0\, g(r_t - \tau_t)\,, \tag{2}$$

where $r_d$ denotes the spatial span of the segment along dimension $d \in \{x, y\}$, and $r_t$ denotes the temporal span. $\boldsymbol{\tau} = (\tau_x, \tau_y, \tau_t)$ are the predefined spatial and temporal span thresholds, and $\lambda_0$ reweights

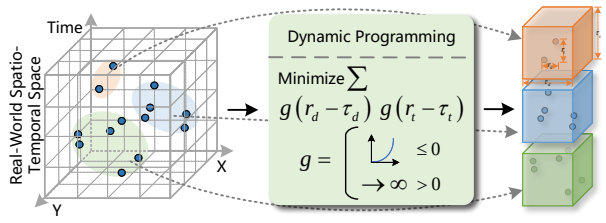



**Figure 3: Candidate segmentation via dynamic programming.**

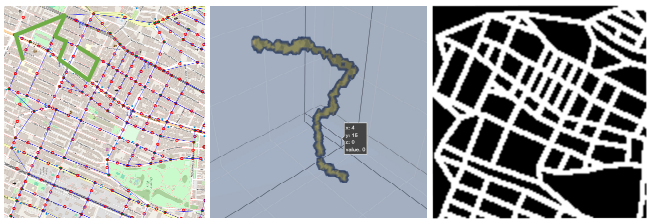
the temporal constraint (spatial weights are set to 1). We use an asymmetric power penalty:

$$g(x) = \begin{cases} x^2, & x \le 0, \\ +\infty, & x > 0. \end{cases} \tag{3}$$

which produces a finite quadratic cost when the segment satisfies the threshold ($r_d \le \tau_d$) and assigns an infinite cost otherwise ($r_d > \tau_d$). This design strictly enforces the region-size constraints to bound computation, and prefers segments whose spans are close to the prescribed limits, avoiding overly small partitions that lead to fragmented recovery.

## 3.2 Spatio-Temporal Voxelization

To recover trajectories, we move beyond the sequence-based representations commonly adopted by prior methods and instead perform trajectory recovery using the voxel representation. To better illustrate the concept, each instance of the voxel representation mentioned below corresponds to a 3D spatio-temporal space. For intuitive understanding, the trajectory imaging representation and road network mask are also visualized in Figure 4.



(a) Trajectory on the Road Network (b) Trajectory Spatio-Temporal Voxelization (c) Road Network Mask

**Figure 4: The example visualization of trajectory representations on a road network.**

We represent trajectory points $[(t_0, x_0, y_0), (t_1, x_1, y_1), \cdots]$ in a spatio-temporal voxel space, where each point is indexed by its location within a voxel grid. By default, we employ nearest-neighbor voxelization to lift the segmented candidates onto the spatio-temporal voxel representation. Note that our formulation is agnostic to the voxel assignment rule, while other assignments can be implemented by distributing each sample to neighboring voxels using kernel-based splatting (e.g., Gaussian kernels) or interpolation-based weighting (e.g., linear or tri-cubic weights). Specifically, each real-world coordinate $\mathbf{p}_i = (t_i, x_i, y_i)$ is transferred to spatio-temporal voxel coordinate $\mathbf{P}_i = (k_i^t, k_i^x, k_i^y)$ via the *linear scaling*,

$$\mathbf{P}_i = \frac{(\mathbf{p}_i - \mathbf{p}_{\min}) \odot \mathbf{v}_{\text{res}}}{(\mathbf{p}_{\max} - \mathbf{p}_{\min})}, \tag{4}$$

where $\mathbf{p}_{\min}$ and $\mathbf{p}_{\max}$ denote the lower and upper bounds of the spatio-temporal region of interest, $\mathbf{v}_{\text{res}} = (T, W, H)$ specifies the voxel resolution, i.e., the number of voxels along the temporal and two spatial axes, and $\odot$ represents element-wise multiplication. The bounds $\mathbf{p}_{\min}$ and $\mathbf{p}_{\max}$ are chosen to cover the target spatio-temporal region, and $\mathbf{v}_{\text{res}}$ is set according to the representation resolution.

The sparse spatio-temporal voxel representation is defined as

$$S = \{(\mathbf{P}_i, v_i)\}_{i=0}^{n-1}, \tag{5}$$

where $n$ denotes the number of voxels with non-zero values, each voxel index $\mathbf{P}_i$ corresponds to a scalar probability value $v_i \in \mathbb{R}$. This representation is analogous to the *coordinate (COO)* format, which is widely used for efficient sparse tensor storage.

The overall process of constructing the spatio-temporal voxel representation is illustrated in Figure 5. Each candidate node is associated with an $(x, y, t)$ coordinate and is transformed into its corresponding voxel location through the aforementioned pipeline, ultimately forming the spatio-temporal voxel representation. Unlike sequential representations that directly rely on numerical values such as position coordinates and timestamps, the spatio-temporal voxel representation encodes the original spatio-temporal information through probabilistic voxelization, where voxel indices capture the underlying spatio-temporal structure.

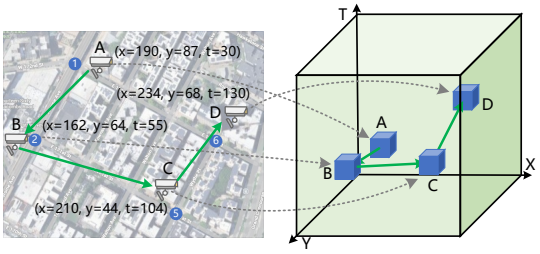



**Figure 5: Spatio-temporal voxelization process visualization.**

This process transforms candidate points into a spatio-temporal voxel representation that seamlessly integrates spatio-temporal information, forming a consistent input representation for deep learning models. Expressing trajectories in volumetric form facilitates the extraction of underlying geographic patterns through neural network learning. This design aligns with *Tobler's First Law of Geography* [21], which states that "everything is related to everything else, but near things are more related than distant things." By jointly encoding spatio-temporal modalities, the volumetric representation enables explicit modeling of spatio-temporal structure that traditional sequence-based methods cannot directly capture. Moreover, integrating spatio-temporal and contextual cues at an early stage facilitates efficient multi-modal fusion and natural extraction of joint features, thereby supporting road-constrained trajectory recovery that incorporates both road network topology and camera-based spatio-temporal signals.

### 3.3 Trajectory Node Extraction

After the voxel-based processing, we obtain the volumetric representation of the reconstructed trajectory. The next step is to extract the sequential trajectory representation from this volumetric form. The core of our trajectory extraction also lies in the transformation between coordinates within the spatio-temporal voxel grid and their corresponding real-world spatio-temporal coordinates. (See Algorithm in Appendix A.2)

We first traverse all road nodes and convert each node $rid$ with real-world spatial position $(x_{rid}, y_{rid})$ to its corresponding spatial voxel coordinates $(k^x_{rid}, k^y_{rid})$:

$$k^x_{rid} = \frac{x_{rid} - x_{\min}}{x_{\max} - x_{\min}} \cdot W, \quad k^y_{rid} = \frac{y_{rid} - y_{\min}}{y_{\max} - y_{\min}} \cdot H, \tag{6}$$

where $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$ represent the spatial bounds.

Next, spatial boundary and probability-based filtering are conducted to identify candidate trajectory points. For each node, we check whether its voxelized coordinates fall within the valid cropped region,

$$0 \le k^x_{rid} < W, \quad 0 \le k^y_{rid} < H. \tag{7}$$

For nodes with valid spatial locations, we collect their voxel activation probabilities $\mathcal{V}(k^t, k^x_{rid}, k^y_{rid})$ across all temporal slices $k^t = 0, \ldots, T-1$, and retain those time steps whose activation values exceed a predefined threshold $\theta = 0.1$. The resulting set of trajectory candidates is defined as

$$\mathcal{M} = \left\{ (rid, k^t) \;\middle|\; \begin{array}{l} 0 \le k^t < T, \\ 0 \le k^x_{rid} < W, \quad 0 \le k^y_{rid} < H, \\ \mathcal{V}(k^t, k^x_{rid}, k^y_{rid}) > \theta \end{array} \right\}. \tag{8}$$

The temporal indices of the selected candidates in $\mathcal{M}$ are then transferred back to real-world timestamps via linear interpolation:

$$t = t_{\min} + k^t \cdot \frac{t_{\max} - t_{\min}}{T}, \tag{9}$$

where $t_{\min}$, $t_{\max}$ are the bounds of the temporal window, and $T$ is the temporal dimensional size of the voxel grid.

Trajectories are extracted by identifying spatio-temporally connected components in the estimated voxel representation. Each connected component corresponds to a candidate sub-trajectory, from which the longest temporally ordered path is selected. Specifically, to eliminate false positives caused by adjacent nodes in dense road regions, we employ a depth-first search (DFS) to identify the longest valid sequence of connected nodes in ascending timestamp order, while properly handling connections among nodes with identical timestamps. Since road nodes are extracted from a single connected component within a local spatio-temporal volume, their number is strictly limited, resulting in a short processing time.

Finally, node timestamp alignment is performed to ensure the temporal consistency during the conversion back to sequence representation. The timestamps of the extracted trajectory nodes are matched to those of the original clustered snapshot nodes, and aligned whenever their temporal distance falls below a threshold.

### 3.4 Trajectory Segment Fusion

To reconstruct complete trajectories, we design a fusion algorithm that merges these partial segments based on their spatio-temporal relationships, as shown in Figure 6.

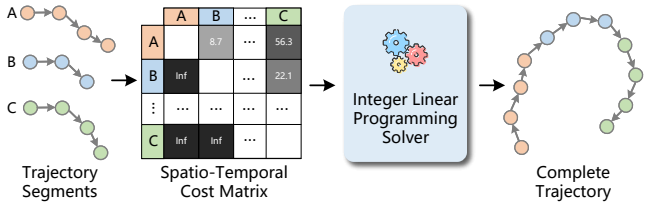



**Figure 6: Trajectory segmentation fusion process.**

We first compute pairwise distances between trajectory segments. For each trajectory pair $(p, q)$, we define a stitching cost $Dist_{pq}$ that quantifies their distance. Let $(rid_p^{(1)}, t_p^{(1)})$ and $(rid_p^{(n_p)}, t_p^{(n_p)})$ denote the start and end nodes of trajectory $\Gamma_p$, and $(rid_q^{(1)}, t_q^{(1)})$, $(rid_q^{(n_q)}, t_q^{(n_q)})$ those of $\Gamma_q$. The cost is computed as:

$$Dist_{pq} = \begin{cases} \alpha\, d(rid_p^{(n_p)}, rid_q^{(1)}) + \beta\, |t_p^{(n_p)} - t_q^{(1)}|, & \text{if } t_p^{(n_p)} < t_q^{(1)}, \\ \alpha\, d(rid_q^{(n_q)}, rid_p^{(1)}) + \beta\, |t_q^{(n_q)} - t_p^{(1)}|, & \text{if } t_q^{(n_q)} < t_p^{(1)}, \\ +\infty, & \text{otherwise.} \end{cases} \tag{10}$$

where $d(\cdot, \cdot)$ is the shortest path distance on the road network, and $\alpha$ and $\beta$ balance the spatial and temporal components of the stitching cost. By default, we set $\alpha = W/(x_{\max} - x_{\min})$ and $\beta = 10 \cdot T/(t_{\max} - t_{\min})$. This cost can be interpreted as using dimension-scaled endpoint distances within our voxel representation, and it can also be defined using other continuity metrics that measure the distance between trajectory pairs.

Given $M$ initial trajectories $\{\Gamma_1, \Gamma_2, \ldots, \Gamma_M\}$, our objective is to automatically merge a subset of them into a new trajectory set that minimizes the total stitching cost while significantly reducing the overall number of trajectories. Only pairs with $Dist_{pq} < \eta$, where $\eta$ is a predefined cost threshold, are considered valid candidates for merging, forming the candidate edge set.

We formulate the trajectory fusion process as the following integer linear programming (ILP) problem:

$$\min_{x,\, y} \left( \sum_{(p,q) \in E} Dist_{pq} x_{pq} + \lambda_1 \sum_{p=1}^{M} y_p \right). \tag{11}$$

Here, $x_{pq} \in \{0, 1\}$ indicates whether trajectory $\Gamma_p$ is stitched to $\Gamma_q$, and $y_p \in \{0, 1\}$ denotes whether $\Gamma_p$ serves as a starting segment in the merged trajectory. The constraints ensure that each trajectory has at most one predecessor and one successor:

$$\begin{aligned} \sum_{q:(p,q) \in E} x_{pq} &\le 1, \quad \forall p, \\ \sum_{q:(q,p) \in E} x_{qp} &\le 1, \quad \forall p, \\ y_p &= 1 - \sum_{q:(q,p) \in E} x_{qp}, \quad \forall p. \end{aligned} \tag{12}$$

The overall workflow includes sorting and preprocessing the initial trajectories, computing all candidate stitching costs $Dist_{pq}$, filtering valid pairs based on the threshold $\eta$, and constructing the ILP model with the above objective and constraints. The optimal

merging scheme is obtained using the efficient ILP solver Gurobi[1]. Finally, the merged trajectories $\{\Pi_1, \ldots, \Pi_K\}$ are reconstructed according to the optimization solution, where each $\Pi_k$ represents an ordered concatenation of initial segments.

After merging, we revalidate the connectivity between consecutive nodes. If a direct connection is missing, the shortest path is generated to bridge the gap under rule-based judgment, ensuring the continuity and reliability of the final reconstructed trajectories.

## 3.5 Model Training and Inference

The proposed framework is flexible and can accommodate different network backbones. By default, it is instantiated with the TransBTS architecture[2], which integrates Transformer modules into a 3D convolutional backbone [25].

To incorporate road network constraints during training, the road topology is projected onto the voxel grid as a binary imaging mask via coordinate transformation, as illustrated in Figure 4(c). Only voxels that fall within the road network regions are considered for loss computation and subsequent trajectory node extraction. This design explicitly restricts the learning process to physically valid spatial regions and provides precise spatial guidance for trajectory generation. The network parameters $\boldsymbol{\phi}$ are optimized using the following Dice loss [17]:

$$\mathcal{L}(\boldsymbol{\phi}) = 1 - \frac{2\sum_{i=1}^{M} v_i \hat{v}_i}{\sum_{i=1}^{M} v_i + \sum_{i=1}^{M} \hat{v}_i + \varepsilon}, \tag{13}$$

where $v_i$ and $\hat{v}_i$ denote the predicted voxel probability and the ground truth value of voxel $i$, respectively, $M$ is the total number of voxels, and $\varepsilon$ is a small constant for numerical stability. The ground truth is obtained by voxelizing real-world road-constrained trajectories within the same spatio-temporal extent (see details of training data generation in Appendix A.3).

In addition, L2 regularization is applied by setting the parameter weight_decay in the Adam optimizer, which is equivalent to adding a penalty term $\lambda\|\boldsymbol{\phi}\|_2^2$ to the loss. After training, the spatio-temporal voxel representation obtained in Section 3.2 is used for inference during trajectory recovery, yielding voxelized recovered trajectories that are subsequently processed in Section 3.3.

# 4 Spatio-Temporal Voxel-based Metric

Motivated by the advantages of spatio-temporal voxel representations, we extend this representation to trajectory evaluation. We next present the formulation of the proposed metric along with the optimization strategy to improve computational efficiency.

*Formulation.* Both predicted and ground truth trajectories are projected into spatio-temporal voxel representations with the same spatial extent and temporal span via Equation (4). Trajectory distance is then quantified by measuring the discrepancy between the two voxelized representations using similarity losses or distance metrics, such as Mean Squared Error (MSE), where smaller values indicate higher similarity. Unless otherwise stated, we adopt MSE as the default metric and refer to it as **Vox-MSE**. This metric is used throughout our experiments.

[1]https://www.gurobi.com

[2]Other backbone architectures can also be used with appropriate configuration.

*Computational Efficiency Optimization.* Sequence-based trajectory comparison typically requires pairwise alignment with a time complexity of $O(mn)$, where $m$ and $n$ are the lengths of two trajectories. In contrast, the voxel-based metric operates directly on the spatio-temporal voxel grid with a complexity of $O(TWH)$, where $T$, $W$, and $H$ denote the temporal and spatial dimensions of the voxel grid. To further improve efficiency, we adopt the sparse representation Equation (5), in which each trajectory is encoded as a set of occupied voxel indices. Distance is then computed through voxel-level overlap rather than dense numerical integration.

When each trajectory is represented as a set of binary voxel indices, the Vox-MSE between two voxelized trajectories $A$ and $B$ can be equivalently expressed using their symmetric difference:

$$|A \triangle B| = |A| + |B| - 2|A \cap B|, \tag{14}$$

where $|A \triangle B|$ denotes the number of mismatched voxels. To account for varying trajectory lengths, we normalize the loss by the total number of occupied voxels,

$$L_{\text{Vox-MSE}} = \frac{|A \triangle B|}{|A| + |B|}. \tag{15}$$

This sparse formulation significantly reduces computational overhead while preserving the fidelity of trajectory distance estimation.

Unlike conventional sequence-based metrics that rely on pointwise correspondence, the proposed voxel-based metric jointly encodes spatial and temporal information, allowing all occupied voxels to contribute to the similarity computation. Discrepancies between distant trajectory nodes span a larger spatio-temporal volume and thus incur higher penalties, enabling the metric to capture both path geometry and node significance. In summary, the proposed voxel-based metric provides a principled spatio-temporal shape-aware evaluation of trajectory reconstruction accuracy by measuring distance through voxel discrepancy. Moreover, the resulting voxel representation forms a compact spatio-temporal embedding that has the potential to support downstream tasks such as trajectory indexing and retrieval, which we leave for future work.

# 5 Experimental Study

We conduct extensive experiments to evaluate the effectiveness of RETRIEVE and to analyze the properties of the proposed metric. We aim to investigate the following research questions:

- **RQ1**: How does the performance of the proposed algorithm compare with different baselines in trajectory recovery tasks?
- **RQ2**: What are the properties of the Vox-MSE metric that facilitate its utilization in trajectory analysis?
- **RQ3**: How does voxel resolution affect recovery accuracy, computational cost, and memory consumption?
- **RQ4**: How does the robustness of the proposed algorithm against camera-road matching errors compare to baselines?
- **RQ5**: What are the resource overheads incurred by the proposed algorithm and metric?
- **RQ6**: How does RETRIEVE visually outperform the baseline?

## 5.1 Experimental Setting

*5.1.1 Datasets.* To evaluate the proposed algorithm, we conduct experiments on three datasets, including one real-world dataset

**Table 1: Overall performance of road-constrained trajectory recovery (↑ indicates higher is better, ↓ indicates lower is better).**

| Method | UrbanVehicle | | | | Carla | | | | Video-SG | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | F1-score ↑ | LCSS ↑ | STLC ↑ | Vox-MSE ↓ | F1-score ↑ | LCSS ↑ | STLC ↑ | Vox-MSE ↓ | F1-score ↑ | LCSS ↑ | STLC ↑ | Vox-MSE ↓ |
| VeTrac [22] | 0.6445 | 0.5665 | 0.7791 | **0.6355** | 0.3989 | 0.2479 | 0.7168 | 0.9692 | 0.5344 | 0.3973 | 0.6480 | 0.6912 |
| MMVC+ [32] | 0.6613 | 0.6901 | 0.8374 | 0.6891 | 0.5220 | 0.4088 | **0.8398** | 0.8973 | 0.5431 | 0.4315 | 0.6722 | 0.7257 |
| MMVC+-C [33] | 0.6751 | 0.7434 | 0.8629 | 0.6721 | 0.5044 | 0.3607 | 0.7406 | 0.9661 | 0.5338 | 0.4359 | 0.6725 | 0.7226 |
| TrajRecovery [29] | 0.6783 | 0.5208 | 0.7573 | 0.6642 | 0.3144 | 0.2540 | 0.7252 | 0.9242 | 0.5490 | 0.4226 | 0.6952 | 0.6807 |
| RETRIEVE | **0.7604** | **0.6971** | **0.8656** | 0.6508 | **0.6749** | **0.6217** | 0.8093 | **0.6114** | **0.5639** | **0.4972** | **0.7523** | **0.6099** |

and two large-scale synthetic datasets. *UrbanVehicle*[3] from previous work [18] provides a publicly available real-world dataset for trajectory recovery. Following the methodology and ground truth trajectories provided in [8], we further curate the dataset through visualization and manual verification for this task. In addition, we use two synthetic datasets. *Carla* [9] contains 1,169 surveillance videos generated in a simulated metropolitan environment using the CARLA[4] engine [10], with virtual cameras deployed over realistic road networks. *Video-SG* [8] is generated by deploying 200 cameras over a subregion of the Singapore road network to produce synthetic surveillance videos for evaluation.

*5.1.2 Comparison Algorithms.* We compare our method against representative approaches, including VeTrac [22], MMVC+ [32], MMVC+-C [33], and TrajRecovery [29]. In our implementation, all methods uniformly use the GPU-based clustering module from MMVC+. For VeTrac/MMVC+/MMVC+-C algorithms, clustering is performed on the final updated snapshot embeddings, after which the shortest-path completion is applied to recover the complete trajectories. Both our approach and TrajRecovery employ GPU-accelerated deep neural modules as the core component for road-constrained trajectory recovery. Specifically, TrajRecovery utilizes a neural network to complete road segments between temporally adjacent nodes within each cluster. Appendix B.1 provides additional details on the algorithm settings.

*5.1.3 Evaluation Metrics.* Referring to previous studies, we utilize similarity metrics LCSS [23] and STLC [20] to evaluate trajectory recovery performance. The ground truth trajectory for each vehicle is represented as $[(rid_0, TS_0, TE_0), (rid_1, TS_1, TE_1), \cdots]$, where $(TS_i, TE_i)$ represent the corresponding start and end timestamps. A recovered road node is regarded as a correct match if its ID is identical to the ground truth and the associated timestamp falls within the corresponding $[TS, TE]$ interval. Based on this matching rule, we also compute precision, recall, and F1-score for each recovered trajectory. Additionally, we further assess the algorithm's performance using our proposed Vox-MSE.

## 5.2 Overall Performance (RQ1)

To comprehensively evaluate overall performance, we compare RETRIEVE with representative trajectory recovery baselines. Table 1 summarizes the comparative results across datasets.

Overall, RETRIEVE outperforms existing methods across datasets on most metrics. By first partitioning trajectory candidates under spatio-temporal constraints and subsequently recovering trajectories based on the effective spatio-temporal voxel representation, RETRIEVE is able to better disentangle erroneously mixed trajectories. Furthermore, the proposed trajectory segment fusion module effectively merges fragmented segments, leading to more complete trajectory recovery.

Baseline methods rely on explicitly matching camera nodes to road nodes or edges during their reconstruction pipelines. Such designs inevitably accumulate camera-road matching errors, which propagate and degrade final recovery accuracy. In contrast, RETRIEVE learns the relationship between camera candidates and road-constrained trajectories within the voxel representation, thereby mitigating the errors introduced by explicit camera-road matching.

In addition, VeTrac, MMVC+, and MMVC+-C reconstruct trajectories based on heuristic rules, which often yield suboptimal results. Although TrajRecovery adopts a deep learning module, its reliance on adjacent snapshots and the lack of noise suppression limit recovery quality. Moreover, its module targets sparse scenarios where cameras are far fewer than road nodes; on Carla, however, cameras outnumber road nodes, rendering this design ineffective.

Performance variations across datasets can be attributed to their distinct characteristics. VeTrac achieves relatively high Vox-MSE scores on UrbanVehicle by disconnecting low-probability node pairs, effectively eliminating spatiotemporally distant noise. This advantage is amplified by the shape-aware nature of Vox-MSE. MMVC+ achieves relatively high STLC scores on Carla because its clustering leverages road network priors to bridge breakpoints into longer trajectories. These longer trajectories, with most nodes lying in nearby regions, still yield high STLC scores despite local topological errors.

## 5.3 Voxel-based Metric Analysis (RQ2)

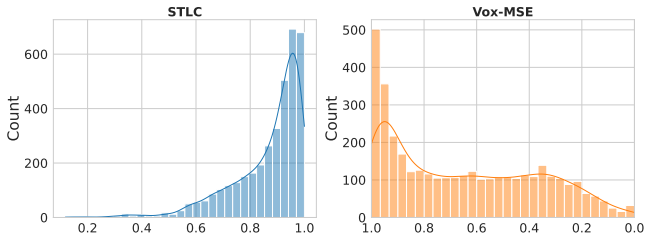



**Figure 7: Metric score distributions on UrbanVehicle dataset.**

To better understand the evaluation characteristics of different trajectory metrics, we visualize the per-trajectory score distributions of different metrics in Figure 7. All metrics are normalized to the range $[0, 1]$. Notably, our metric evaluates the distance between trajectories, where smaller values indicate higher similarity,

[3]https://github.com/tsinghua-fib-lab/MMVC_de_com
[4]https://carla.org/

whereas the other metrics adopt the opposite convention. To facilitate the comparison, we reverse the horizontal axis for Vox-MSE so that values closer to the right consistently indicate higher trajectory similarity across all subplots. These metrics exhibit notably different score distributions on the UrbanVehicle dataset. Our shape-aware metric explicitly encodes the spatio-temporal information of entire trajectories, which leads to evaluation value distributions that differ from those of existing metrics. The distinct distribution of Vox-MSE suggests that it captures evaluation aspects not reflected by conventional metrics, providing complementary and informative signals for assessing trajectory recovery quality. (Supplementary details in Appendix B.2)

## 5.4 Voxel Resolution Analysis (RQ3)

We conduct experiments on the UrbanVehicle dataset to examine the trade-off between recovery accuracy and computational cost under varying voxel resolutions.

As shown in Figure 8(a), lowering the voxel resolution (i.e., using coarser voxels) reduces time cost but degrades recovery accuracy. The F1-score drops sharply once the resolution falls below $64 \times 64 \times 64$, indicating that overly coarse voxels fail to preserve the fine-grained spatio-temporal patterns necessary for accurate recovery. Time cost also decreases with resolution, declining steeply at high resolutions before leveling off at lower ones.

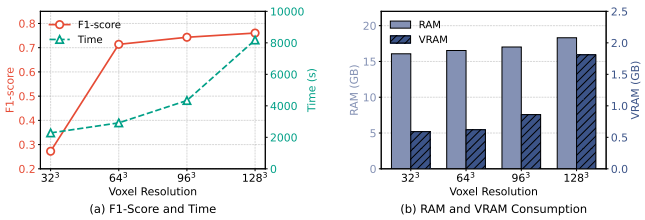



**Figure 8: Impact of voxel resolutions on trajectory recovery accuracy and resource consumption.**

As shown in Figure 8(b), RAM and VRAM measured during inference (excluding the clustering phase) exhibit distinct scaling behaviors. RAM usage grows slowly and remains relatively stable. VRAM grows approximately cubically, since the 3D voxel grid of size $T \times W \times H$ must be fully instantiated in GPU memory. We also observe that, under a fixed resolution, varying the module design yields nearly identical memory footprints, confirming that voxel resolution is the dominant factor governing VRAM consumption. Therefore, a higher resolution is recommended when VRAM and time budgets permit.

## 5.5 Sensitivity Analysis of Camera-Road Matching Errors (RQ4)

To simulate the impact of camera-road matching errors on trajectory recovery, we inject uniformly distributed noise of up to 20 meters into camera positions and observe the resulting performance variations, as shown in Figure 9 and Appendix B.3. As the noise magnitude increases, our method consistently outperforms baselines across all metrics. Although the Vox-MSE metric is generally more sensitive to errors in high-quality recovered trajectories, our method consistently demonstrates the best performance.

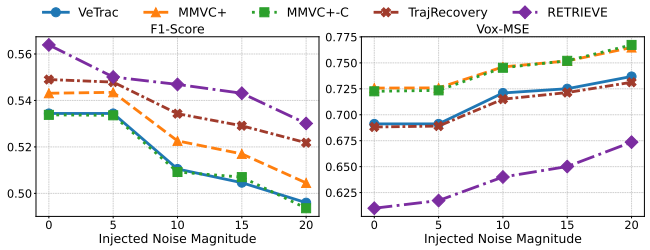



**Figure 9: Performance with varying injected noise.**

## 5.6 Overhead Analysis (RQ5)

We analyze the computational overhead of both the proposed trajectory recovery algorithm and the evaluation metric, including memory usage, GPU memory consumption, and runtime.

*5.6.1 Trajectory Recovery Overhead.* We report the memory and time overhead of the trajectory recovery algorithms to characterize their computational efficiency. Only the resource consumption during inference is reported. (Additional details in Appendix B.4)

**Memory Overhead.** As illustrated in Figure 10, the peak memory consumption of RETRIEVE is relatively low. This efficiency results from processing and predicting trajectories for each cluster independently, which effectively reduces peak memory usage.

**GPU Memory Overhead.** Figure 11 compares the GPU memory consumption across all methods. Our algorithm exhibits moderate GPU memory usage relative to the baseline algorithms.

**Time Overhead.** As shown in Figure 12, RETRIEVE achieves the state-of-the-art computational efficiency among all methods. This improvement stems from its framework design, which directly converts the camera observations into the road-constrained trajectory voxel representation, allowing the road sequence to be extracted without additional iterative processing.

*5.6.2 Evaluation Metric Overhead.* We further evaluate the computational overhead of the trajectory evaluation metric. By default, the evaluation of all metrics is conducted on the recovery results produced by RETRIEVE. In our implementation, all evaluation metrics are computed *without GPU* acceleration.

**Memory Overhead.** The memory usage of all trajectory evaluation metrics is nearly identical, as each candidate trajectory is independently matched to its corresponding ground truth, resulting in a consistently low memory footprint across metrics.

**Time Overhead.** Execution times for trajectory evaluation under different metrics are summarized in Figure 13. F1-score evaluation achieves the lowest execution time due to dictionary-based lookups for node matching. Among metrics that incorporate temporal context, Vox-MSE exhibits execution time comparable to that of STLC.

## 5.7 Trajectory Visualization Analysis (RQ6)

We visualize a representative recovered trajectory in Figure 14 to compare our method with MMVC+. As observed, MMVC+ fails to filter a noisy snapshot, producing a false positive that leads to incorrect trajectory branches. This occurs because the noisy

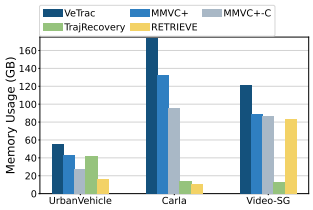



**Figure 10: Algorithm memory usage comparison.**

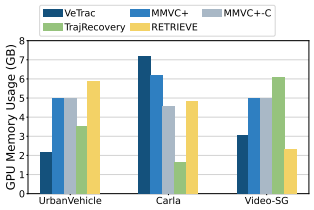



**Figure 11: Algorithm GPU memory usage comparison.**

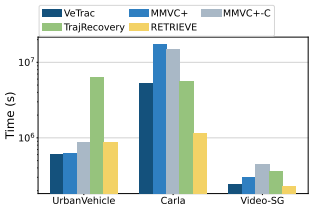



**Figure 12: Algorithm efficiency comparison.**

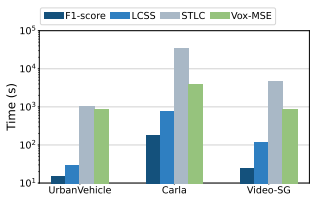



**Figure 13: Metric efficiency comparison.**

sample is visually similar to valid observations and lies along a high-likelihood arterial road. In contrast, our method suppresses such noise and recovers a trajectory consistent with the ground truth, as our spatio-temporal voxel representation better exploits spatio-temporal information to achieve higher recovery accuracy.

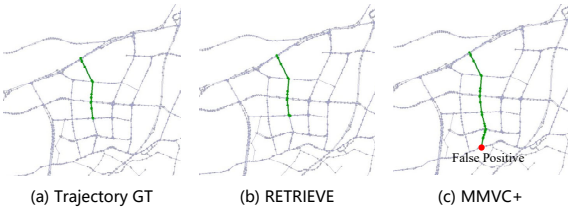



**Figure 14: Ground truth and recovered trajectories from RETRIEVE and MMVC+.**

# 6 Related Work

## 6.1 Trajectory Imaging Representation

Early studies [27, 28] propose encoding time series as images to enable computer vision techniques for classification and imputation, offering compact representation and effective modeling of temporal dependencies. Following this paradigm, trajectories can be viewed as multi-modal spatio-temporal time series and transformed into image representations [1]. One common approach is rasterization, where data are converted into raster images, enabling trajectories to be represented as binary or multi-channel images. In its simplest form, pixels traversed by a trajectory are assigned nonzero values. More advanced rasterization schemes embed richer spatio-temporal semantics. Endo et al. [11] encode GPS trajectories into images where pixel intensities reflect dwell time, enabling transportation mode inference. Yang and Gidófalvi [31] introduce directional flow images, a multi-channel representation capturing local motion directions, which are combined with deep learning models to identify regional movement patterns. TrjSR [4] proposes a generative framework for trajectory similarity by reconstructing super-resolution trajectory images from low-resolution inputs. Chang et al. [6] leverage binary trajectory images to compute trajectory similarity, integrating the advantages of both sequence-based and image-based representations. Readers may refer to the comprehensive overview of imaging strategies in [1, 19]. Distinct from these imaging-based approaches, our work adopts a spatio-temporal voxel representation tailored for trajectory reconstruction, which better preserves spatial continuity and temporal coherence.

## 6.2 Trajectory Recovery from Videos

Vehicle trajectory recovery from camera networks was first explored in MMVC [18] and VeTrac [22]. MMVC constructs snapshot clusters using visual similarity and license plate text similarity, followed by iterative merge-and-denoise refinement under spatio-temporal constraints. Clusters are merged based on LCSS similarity of recovered trajectories, while outliers are progressively removed. VeTrac [22] extends snapshot similarity by incorporating mobility probability modeled via inverse Gaussian distributions. A similarity graph is constructed and embedded using a graph convolutional network, and snapshots are clustered using HDBSCAN [3]. MMVC+ [32] improves this paradigm by replacing heuristic rules with probabilistic estimation and enabling dynamic feature adaptation during iterative refinement, though at the cost of high computational overhead. MMVC+-C [33] further optimizes efficiency by partitioning the map into regions and recovering trajectories independently before aggregation. More recently, TrajRecovery [29] employs deep learning and spatial transfer probability models, introducing a Motion-GAT module to model urban road constraints and an OD-Attention mechanism to capture driver-specific behavior.

# 7 Conclusion

In this work, we propose RETRIEVE, a novel framework that integrates a spatio-temporal voxel representation with a segment-fusion mechanism to recover vehicle road-constrained trajectories from large-scale video data. By modeling trajectories within the spatio-temporal voxel space, RETRIEVE effectively captures spatial structures and temporal continuity, while avoiding explicit camera-road matching and thereby mitigating error propagation. Furthermore, we introduce Vox-MSE, a new evaluation metric that quantifies trajectory recovery quality within the spatio-temporal voxel grid context, offering a complementary perspective to existing evaluation approaches. Extensive experiments demonstrate that RETRIEVE outperforms existing trajectory recovery approaches while maintaining competitive computational efficiency.


# Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (Grant No. 82502508), in part by the Gansu Provincial Key Research and Development Program (Grant No. 25YFWA005), in part by the Youth Science and Technology Talent Innovation Project of Lanzhou City (Grant No. 2025-QN-037), and in part by the Major Projects of Zhejiang Province (Grant No. LD24F020012).

## References


[1] Duygu Altunkaya, Feyza Yildirim Okay, and Suat Özdemir. 2023. Image Transformation for IoT Time-Series Data: A Review. *CoRR* abs/2311.12742 (2023). arXiv:2311.12742 doi:10.48550/ARXIV.2311.12742

[2] Yijun Bei, Teng Ma, Dongxiang Zhang, Sai Wu, Kian-Lee Tan, and Gang Chen. 2025. Mining Platoon Patterns from Traffic Videos. *Proc. VLDB Endow.* 18, 6 (2025), 1839–1851. doi:10.14778/3725688.3725710

[3] Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-Based Clustering Based on Hierarchical Density Estimates. In *Advances in Knowledge Discovery and Data Mining, 17th Pacific-Asia Conference, PAKDD 2013, Gold Coast, Australia, April 14-17, 2013, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 7819)*, Jian Pei, Vincent S. Tseng, Longbing Cao, Hiroshi Motoda, and Guandong Xu (Eds.). Springer, Gold Coast, Australia, 160–172. doi:10.1007/978-3-642-37456-2_14

[4] Hanlin Cao, Haina Tang, Yulei Wu, Fei Wang, and Yongjun Xu. 2021. On Accurate Computation of Trajectory Similarity via Single Image Super-Resolution. In *International Joint Conference on Neural Networks, IJCNN 2021, Shenzhen, China, July 18-22, 2021*. IEEE, Shenzhen, China, 1–9. doi:10.1109/IJCNN52387.2021.9533802

[5] Zijian Cao, Dong Zhao, Hanxing Song, Haitao Yuan, Qiyue Wang, Huadong Ma, Jianjun Tong, and Chang Tan. 2024. F$^{3}$VeTrac: Enabling Fine-Grained, Fully-Road-Covered, and Fully-Individual- Penetrative Vehicle Trajectory Recovery. *IEEE Trans. Mob. Comput.* 23, 5 (2024), 4975–4991. doi:10.1109/TMC.2023.3301871

[6] Zhihao Chang, Linzhu Yu, Huan Li, Sai Wu, Gang Chen, and Dongxiang Zhang. 2024. Revisiting CNNs for Trajectory Similarity Learning. *Proc. VLDB Endow.* 18, 4 (2024), 1013–1021. https://www.vldb.org/pvldb/vol18/p1013-chang.pdf

[7] Lei Chen, M. Tamer Özsu, and Vincent Oria. 2005. Robust and Fast Similarity Search for Moving Object Trajectories. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, Fatma Özcan (Ed.). ACM, Baltimore, Maryland, USA, 491–502. doi:10.1145/1066157.1066213

[8] Taihang Dong, Dingyu Yang, Ping Chen, and Dongxiang Zhang. 2025. Query-Aware Path Inference from Spatial Videos. *Proc. ACM Manag. Data* 3, 6, Article 352 (Dec. 2025), 26 pages. doi:10.1145/3769817

[9] Taihang Dong, Dingyu Yang, Dongping Zhang, Sai Wu, Shaojie Qiao, and Dongxiang Zhang. 2026. Efficient and Accurate Cross-Camera Vehicle Trajectory Recovery . *IEEE Transactions on Knowledge & Data Engineering* 38, 02 (Oct. 2026), 1–16. doi:10.1109/TKDE.2025.3619943

[10] Alexey Dosovitskiy, Germán Ros, Felipe Codevilla, Antonio M. López, and Vladlen Koltun. 2017. CARLA: An Open Urban Driving Simulator. In *1st Annual Conference on Robot Learning, CoRL 2017, Mountain View, California, USA, November 13-15, 2017, Proceedings (Proceedings of Machine Learning Research, Vol. 78)*. PMLR, Mountain View, California, USA, 1–16. http://proceedings.mlr.press/v78/dosovitskiy17a.html

[11] Yuki Endo, Hiroyuki Toda, Kyosuke Nishida, and Jotaro Ikedo. 2016. Classifying spatial trajectories using representation learning. *Int. J. Data Sci. Anal.* 2, 3-4 (2016), 107–117. doi:10.1007/S41060-016-0014-1

[12] Robert J. Fowler, Mike Paterson, and Steven L. Tanimoto. 1981. Optimal Packing and Covering in the Plane are NP-Complete. *Inf. Process. Lett.* 12, 3 (1981), 133–137. doi:10.1016/0020-0190(81)90111-3

[13] Lingxiao He, Xingyu Liao, Wu Liu, Xinchen Liu, Peng Cheng, and Tao Mei. 2023. FastReID: A Pytorch Toolbox for General Instance Re-identification. In *Proceedings of the 31st ACM International Conference on Multimedia, MM 2023, Ottawa, ON, Canada, 29 October 2023- 3 November 2023*, Abdulmotaleb El-Saddik, Tao Mei, Rita Cucchiara, Marco Bertini, Diana Patricia Tobon Vallejo, Pradeep K. Atrey, and M. Shamim Hossain (Eds.). ACM, Ottawa, ON, Canada, 9664–9667. doi:10.1145/3581783.3613460

[14] Gabriel Hiribarren and Juan Carlos Herrera. 2014. Real time traffic states estimation on arterials based on trajectory data. *Transportation Research Part B: Methodological* 69 (2014), 19–30.

[15] Weiming Hu, Xi Li, Guodong Tian, Stephen J. Maybank, and Zhongfei Zhang. 2013. An Incremental DPMM-Based Method for Trajectory Clustering, Modeling, and Retrieval. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 5 (2013), 1051–1065.

[16] Li Li, Rui Jiang, Zhengbing He, Xiqun Michael Chen, and Xuesong Zhou. 2020. Trajectory data-based traffic flow studies: A revisit. *Transportation Research Part C: Emerging Technologies* 114 (2020), 225–240.

[17] Xiaoya Li, Xiaofei Sun, Yuxian Meng, Junjun Liang, Fei Wu, and Jiwei Li. 2020. Dice Loss for Data-imbalanced NLP Tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault (Eds.). Association for Computational Linguistics, Online, 465–476. doi:10.18653/V1/2020.ACL-MAIN.45

[18] Zongyu Lin, Guozhen Zhang, Zhiqun He, Jie Feng, Wei Wu, and Yong Li. 2021. Vehicle Trajectory Recovery on Road Network Based on Traffic Camera Video Data. In *GIS*, Xiaofeng Meng, Fusheng Wang, Chang-Tien Lu, Yan Huang, Shashi Shekhar, and Xing Xie (Eds.). ACM, Beijing, 389–398.

[19] Jingchao Ni, Ziming Zhao, ChengAo Shen, Hanghang Tong, Dongjin Song, Wei Cheng, Dongsheng Luo, and Haifeng Chen. 2025. Harnessing Vision Models for Time Series Analysis: A Survey. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025*. ijcai.org, Montreal, Canada, 10612–10620. doi:10.24963/IJCAI.2025/1178

[20] Shuo Shang, Lisi Chen, Zhewei Wei, Christian S. Jensen, Kai Zheng, and Panos Kalnis. 2017. Trajectory Similarity Join in Spatial Networks. *Proc. VLDB Endow.* 10, 11 (2017), 1178–1189. doi:10.14778/3137628.3137630

[21] Waldo R Tobler. 1970. A computer movie simulating urban growth in the Detroit region. *Economic geography* 46, sup1 (1970), 234–240.

[22] Panrong Tong, Mingqian Li, Mo Li, Jianqiang Huang, and Xiansheng Hua. 2021. Large-scale vehicle trajectory reconstruction with camera sensing network. In *ACM MobiCom '21: The 27th Annual International Conference on Mobile Computing and Networking, New Orleans, Louisiana, USA, October 25-29, 2021*. ACM, New Orleans, 188–200.

[23] Michail Vlachos, Dimitrios Gunopulos, and George Kollios. 2002. Discovering Similar Multidimensional Trajectories. In *Proceedings of the 18th International Conference on Data Engineering, San Jose, CA, USA, February 26 - March 1, 2002*, Rakesh Agrawal and Klaus R. Dittrich (Eds.). IEEE Computer Society, San Jose, CA, USA, 673–684. doi:10.1109/ICDE.2002.994784

[24] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. 2023. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, Vancouver, BC, Canada, 7464–7475. doi:10.1109/CVPR52729.2023.00721

[25] Wenxuan Wang, Chen Chen, Meng Ding, Hong Yu, Sen Zha, and Jiangyun Li. 2021. TransBTS: Multimodal Brain Tumor Segmentation Using Transformer. In *Medical Image Computing and Computer Assisted Intervention - MICCAI 2021 - 24th International Conference, Strasbourg, France, September 27 - October 1, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12901)*, Marleen de Bruijne, Philippe C. Cattin, Stéphane Cotin, Nicolas Padoy, Stefanie Speidel, Yefeng Zheng, and Caroline Essert (Eds.). Springer, Strasbourg, France, 109–119. doi:10.1007/978-3-030-87193-2_11

[26] Zuchao Wang, Min Lu, Xiaoru Yuan, Junping Zhang, and Huub van de Wetering. 2013. Visual Traffic Jam Analysis Based on Trajectory Data. *IEEE Trans. Vis. Comput. Graph.* 19, 12 (2013), 2159–2168.

[27] Zhiguang Wang and Tim Oates. 2015. Encoding time series as images for visual inspection and classification using tiled convolutional neural networks. In *Workshops at the twenty-ninth AAAI conference on artificial intelligence*. AAAI Press, Palo Alto, California, USA.

[28] Zhiguang Wang and Tim Oates. 2015. Imaging Time-Series to Improve Classification and Imputation. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, Qiang Yang and Michael J. Wooldridge (Eds.). AAAI Press, Buenos Aires, Argentina, 3939–3945. http://ijcai.org/Abstract/15/553

[29] Dongen Wu, Ziquan Fang, Qichen Sun, Lu Chen, Haiyang Hu, Fei Wang, and Yunjun Gao. 2024. TrajRecovery: An Efficient Vehicle Trajectory Recovery Framework based on Urban-Scale Traffic Camera Records. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, Ricardo Baeza-Yates and Francesco Bonchi (Eds.). ACM, Barcelona, Spain, 5979–5990. doi:10.1145/3637528.3671558

[30] Dong Xie, Feifei Li, and Jeff M. Phillips. 2017. Distributed Trajectory Similarity Search. *Proc. VLDB Endow.* 10, 11 (2017), 1478–1489.

[31] Can Yang and Gyözö Gidófalvi. 2020. Detecting regional dominant movement patterns in trajectory data with a convolutional neural network. *Int. J. Geogr. Inf. Sci.* 34, 5 (2020), 996–1021. doi:10.1080/13658816.2019.1700510

[32] Fudan Yu, Wenxuan Ao, Huan Yan, Guozhen Zhang, Wei Wu, and Yong Li. 2022. Spatio-Temporal Vehicle Trajectory Recovery on Road Network Based on Traffic Camera Video Data. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, Washington DC, 4413–4421.

[33] Fudan Yu, Huan Yan, Rui Chen, Guozhen Zhang, Yu Liu, Meng Chen, and Yong Li. 2023. City-scale vehicle trajectory data from traffic camera videos. *Scientific data* 10, 1 (2023), 711.

[34] Dongxiang Zhang, Teng Ma, Junnan Hu, Yijun Bei, Kian-Lee Tan, and Gang Chen. 2023. Co-movement Pattern Mining from Videos. *Proc. VLDB Endow.* 17, 3 (2023), 604–616.

# A Additional Methodology Details

## A.1 Preprocessing and Clustering Details

To provide a clearer understanding of the overall pipeline, we further elaborate on the preprocessing and clustering module, as shown in Figure 15. Video preprocessing focuses on extracting snapshot features from raw videos, including object detection and feature extraction. A clustering algorithm is then applied to group vehicle snapshots based on visual features, assigning a cluster label to each snapshot.

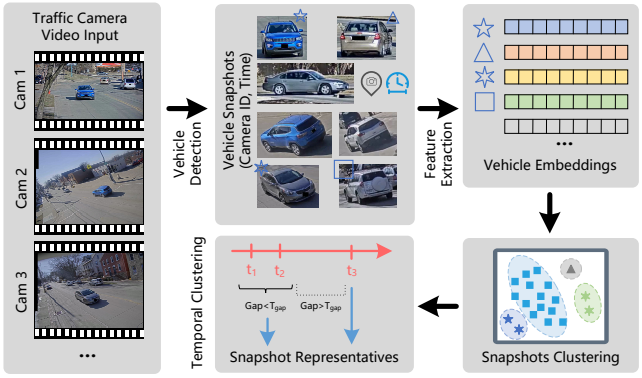



**Figure 15: Video preprocessing and snapshot clustering.**

Specifically, for raw video inputs, preprocessing is required to extract snapshot features. Since the original footage captures all moving objects, we first leverage object detection (e.g., YOLOv7 [24]) to obtain clean vehicle snapshots, and then employ vehicle ReID techniques (e.g., FastReID [13]) to extract appearance features. If bounding boxes or features are directly available in the dataset, they are used as-is. For datasets such as *Carla* [9], directly processing all detected objects from every video frame is cost-prohibitive. To address this, we adopt the one-pass strategy from the TRACER implementation [9] to reduce the number of processed snapshots.

We then apply a clustering algorithm (e.g., MMVC+ clustering [32]) to group all vehicle snapshots, where the similarity between two snapshots is computed using both appearance and plate features. Since license plates are typically more distinctive than vehicle appearances, we assign a higher weight to plate features whenever they are available.

Within each resulting cluster, snapshots are arranged in chronological order, and the temporal merging from VIVID [8] is then applied to consolidate adjacent snapshots into a single representative, further eliminating redundancy. Each cluster then yields a camera trajectory sequence, on which we perform trajectory recovery to derive accurate road-constrained trajectories.

## A.2 Trajectory Node Extraction

To provide a more comprehensive understanding of the trajectory node extraction process, the overall workflow for extracting trajectory sequences from the spatio-temporal voxel representation is summarized in Algorithm 1.

**Algorithm 1** Trajectory Node Extraction

**Input:** Spatio-temporal voxel grid $\mathcal{V}$; Road network node dictionary $\mathcal{N}$.
`// Extract valid candidate road nodes and timestamps`
**for** each node $N_i \in \mathcal{N}$ **do**
  Map $N_i$ to voxel grid coordinates via Equation 6.
  Filter coordinates by spatial bounds and probability via Equation 8.
  Extract corresponding timestamps via Equation 9.
**end for**
`// Extract the longest sub-trajectory`
Segment the voxel grid into connected components $\mathcal{K}$.
**for** each connected component $\mathcal{K}_j$ **do**
  Apply DFS to extract the longest valid trajectory node sequence.
**end for**
**Output:** Final trajectory node sequences.

## A.3 Trajectory Training Data Generation

We construct training data by simulating noisy camera trajectories from ground truth trajectories and aligning them with corresponding road-constrained trajectory supervision. The overall procedure is summarized as follows.

First, we load the ground truth camera trajectories and road-constrained ground truth trajectories, along with the road network and camera locations in the real-world coordinate space. For each camera trajectory, noise is injected by randomly deleting a proportion of trajectory nodes and inserting spurious nodes with randomly sampled timestamps. Throughout this process, an index mapping is maintained to preserve the correspondence between the original ground truth trajectory and the corrupted one.

Next, each ground truth camera trajectory is segmented into multiple continuous sub-trajectories using the candidate segmentation module, i.e., the same dynamic programming-based method under spatio-temporal constraints. The resulting segmentation boundaries are then projected onto the noisy trajectory via the index mapping, ensuring a one-to-one correspondence between ground truth and noisy sub-trajectories.

For each camera sub-trajectory, we further extract the longest continuous road-constrained ground truth sub-trajectory that lies within the corresponding spatio-temporal range. This selection criterion helps prevent incorrect alignment between camera trajectories and unrelated road trajectories. Sub-trajectories with insufficient length are discarded. Finally, the resulting paired samples, each comprising a noisy camera sub-trajectory and the road-constrained ground truth sub-trajectory, are stored as training data.

# B Additional Experimental Details

## B.1 Experimental Setting

*B.1.1 Comparison Algorithm Details.* For algorithms whose source code is not publicly available, we reproduce them following the algorithmic logic described in the original papers. Considering the differences in data distributions across datasets, we apply targeted adaptations to their trajectory recovery inference strategies to maximize their performance on each dataset. For constructing the camera-road correspondence required by certain baselines, we leverage paired camera-level and road-constrained trajectory ground truth to establish the matching for overall performance evaluation.

We additionally provide details on the thread configuration of the compared algorithms. In the official MMVC+ and MMVC+-C implementations, trajectory completion and denoising within large-scale clusters are accelerated via multi-threading to reduce runtime. Similarly, for the reproduction of VeTrac, we enable parallel processing for adjacency matrix construction, which is its most computationally expensive step. TrajRecovery also uses parallel execution to accelerate network inference during road-trajectory generation. Specifically, in our comparative experiments, VeTrac, MMVC+, and MMVC+-C are executed with eight threads, while TrajRecovery is allocated two threads to recover two trajectories simultaneously. Our method processes all trajectories in a single thread, except for the Gurobi-based module which uses four threads.

*B.1.2 Implementation Details.* We report the key hyperparameter settings used across different datasets. For the weighting parameters $\lambda_0$ and $\lambda_1$, we set them to $10^8$ and $10^6$ for UrbanVehicle, $10^{-6}$ and $10^6$ for Carla, and $10^4$ and $10^4$ for Video-SG, respectively. The default spatio-temporal voxel resolution $[W, H, T]$ is set to $[128, 128, 128]$ for UrbanVehicle, $[256, 256, 32]$ for Carla, and $[384, 384, 16]$ for Video-SG. We set $x_{\min}$, $y_{\min}$, $y_{\min}$ to the minimum values among all candidates by default. The corresponding spatio-temporal ranges $[x_{\max} - x_{\min}, y_{\max} - y_{\min}, t_{\max} - t_{\min}]$ are set to $[3000, 3000, 2000]$ for UrbanVehicle, $[2900, 2900, 128]$ for Carla, and $[2900, 2900, 300]$ for Video-SG. For model training, the batch size and learning rate are set to $(3, 10^{-3})$ for UrbanVehicle, $(2, 10^{-4})$ for Carla, and $(2, 10^{-3})$ for Video-SG, respectively.

All the experiments are conducted on a server equipped with 96 CPU cores (Intel(R) Xeon(R) 2.10 GHz), 256GB memory, and one NVIDIA GeForce RTX 4090 Ti GPU.

## B.2 Per-Trajectory Metric Score Distributions

To illustrate the characteristics of different metrics, we visualize a ground truth trajectory within a road network from the UrbanVehicle dataset and a trajectory reconstructed by MMVC+, as shown in Figure 16. The figure also reports the corresponding F1-score, LCSS, STLC, and Vox-MSE values, demonstrating that different metrics respond differently to the same reconstruction outcome.




**Figure 16: The example trajectory case visualization with evaluation metrics.**

We additionally visualize the distribution of evaluation results for the other two metrics, F1-score and LCSS, as shown in Figure 17. On the UrbanVehicle dataset, both F1-score and LCSS exhibit score distributions similar to STLC, yet markedly different from Vox-MSE.

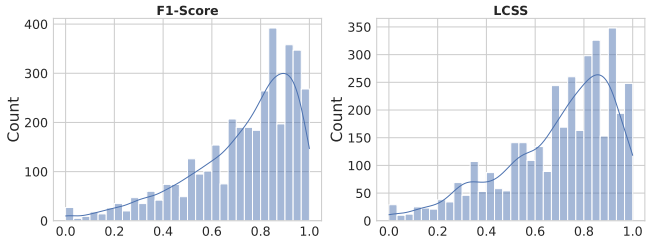



**Figure 17: Metric score distributions on UrbanVehicle dataset.**

## B.3 Sensitivity Analysis of Camera-Road Matching Errors

We additionally analyze the sensitivity of algorithms to camera-road matching errors across all evaluation metrics. Overall, our method consistently achieves accurate trajectory recovery.

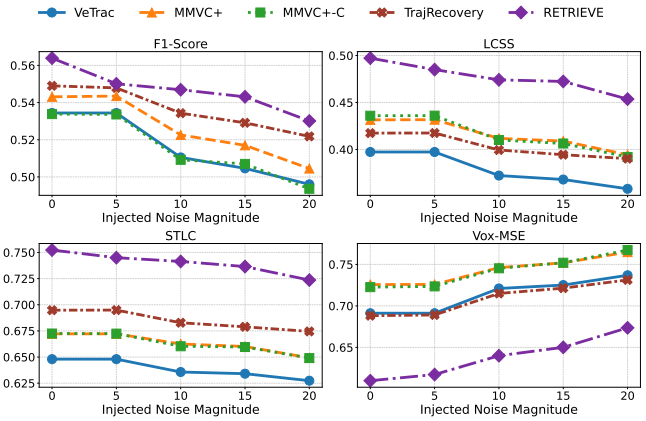



**Figure 18: Performance with varying injected noise.**

## B.4 Overhead Analysis

To better understand the distribution of computational costs across different modules, we report a breakdown of the runtime overhead of RETRIEVE.

**Cost Breakdown** Table 2 summarizes the runtime breakdown of RETRIEVE across different datasets. Overall, the computational cost is primarily dominated by snapshot clustering, spatio-temporal voxelization and model inference, and trajectory node extraction.

**Table 2: RETRIEVE cost breakdown.**

| Module | UrbanVehicle (s) | Carla (s) | Video-SG (s) |
|---|---|---|---|
| Clustering | 747.2 | 4615.2 | 445.9 |
| Candidate Segmentation | 22.5 | 83.0 | 2.4 |
| Voxelization and Model Inference | 4240.4 | 4060.6 | 1024.0 |
| Trajectory Node Extraction | 3209.4 | 2440.4 | 779.6 |
| Trajectory Segment Fusion | 375.2 | 85.2 | 17.8 |